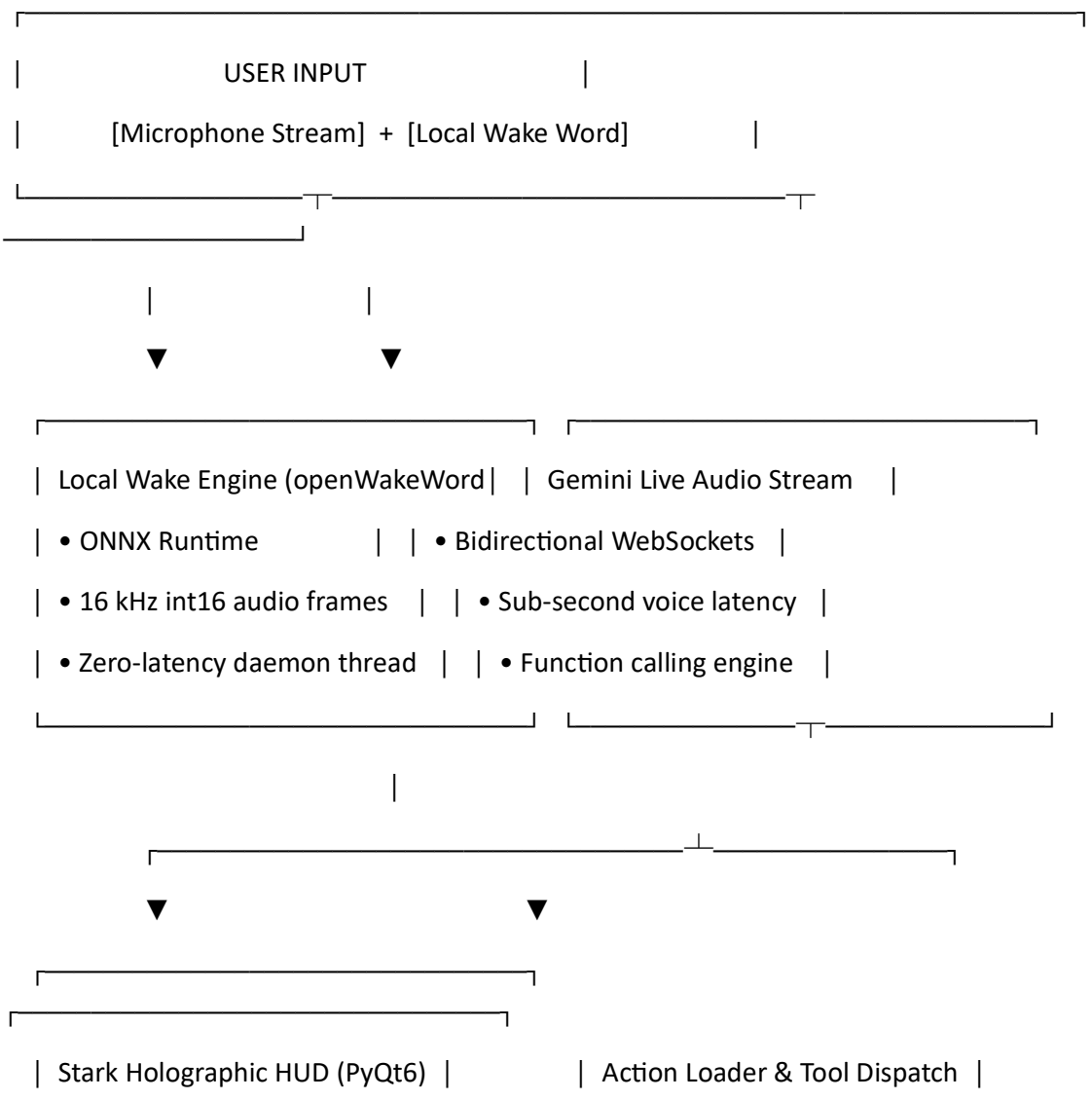
1. The 30-Second Elevator Pitch

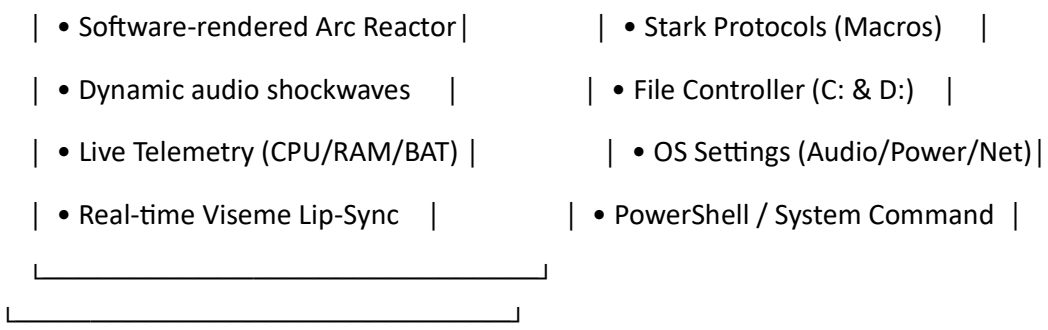
*"BRAVO is an autonomous desktop AI voice assistant inspired by J.A.R.V.I.S. Unlike traditional assistants like Siri or Alexa that rely on rigid, slow 'Speech-to-Text → LLM → Text-to-Speech' pipelines, BRAVO uses a **bidirectional real-time audio WebSocket stream** powered by Google Gemini Live. It features **offline local wake-word detection**, a **custom PyQt6 holographic Arc Reactor HUD** with live hardware telemetry, and an **agentic tool execution engine** that gives it direct, safe control over the laptop's operating system, hardware settings, and file system."*

---

2. The 4 Technical Pillars (System Architecture)

Explain the project by breaking it into **4 modular subsystems**:





Pillar 1: Low-Latency Speech Pipeline (Gemini Live API)

- **What to say:**
    - *"Instead of the conventional 3-step pipeline (Whisper STT → GPT text → ElevenLabs TTS) which has 3 to 5 seconds of lag, I used Google's Gemini Live bidirectional audio streaming."*
    - *"Microphone audio is captured at 16 kHz PCM and piped over WebSockets directly to the model, which streams back raw 24 kHz audio chunks in real time (sub-second latency)."*
    - *"I implemented an EchoGuard filter with latency compensation so the assistant never accidentally interrupts or listens to its own voice."*

Pillar 2: Local & Offline Wake Word Detection (openWakeWord)

- **What to say:**
    - *"For privacy and zero cloud cost when idle, the wake word runs **100% locally and offline** using openWakeWord with ONNX Runtime."*
    - *"It operates in its own dedicated background thread. Microphone frames are fed non-blockingly via an in-memory queue, so wake-word processing never delays the live audio pipeline."*

Pillar 3: Holographic HUD & Hardware Telemetry (PyQt6)

- **What to say:**
    - *"The frontend is built entirely in PyQt6 with custom QPainter graphics—no heavy web frameworks or Electron."*
    - *"It features an animated Stark Arc Reactor with kinetic rotating magnetic coils, hexagonal unibeam geometry, and real-time audio shockwaves that react dynamically to the live frequency level."*

- *"The side panel polls real-time hardware telemetry (psutil, pynvml, wmi) tracking CPU, RAM, Network throughput, GPU, temperatures, and battery power status."*

Pillar 4: Autonomous OS & Multi-Action Engine (actions/)

- **What to say:**
  - *"I built a modular **auto-discovery action system** (core/action_loader.py). Any tool placed in the actions/ folder is inspected via function declaration schemas and registered into Gemini's tool registry at boot."*
  - *"Key capabilities include:*
    - ***Stark Protocols:** One-phrase multi-task macros (e.g., Protocol Focus, Stealth, Overclock/Gaming, Sleep, Status Report).*
    - ***File Controller:** Full file management across both C: and D: drives (searching, reading, organizing) with smart guards that protect critical Windows system folders.*
    - ***Hardware Controls:** Direct OS integration for volume, brightness, window management, power schemes, and PowerShell execution.*"

---

3. Key Challenges & How You Solved Them (Mentor Favorites)

If your mentor asks: *"What was the hardest part or biggest challenge?"* Mention these three points:

1. **Audio Echo & Self-Interruption:**
   - *Problem:* A real-time voice assistant can hear its own speakers and trigger infinite hallucination loops.
   - *Solution:* Built an audio feedback lock and EchoGuard tail calculation (output latency + echo tail) that gates the microphone while the assistant is speaking.
2. **Audio-UI Lip-Sync & Waveform Physics:**
   - *Problem:* Audio packets arrive in chunks every few milliseconds, causing jittery animations.
   - *Solution:* Implemented exponential moving average smoothing (_amp_disp) and viseme timing schedules so visual shockwaves and waveforms glide smoothly at 60 FPS.
3. **Safe System Automation:**

- *Problem:* Giving an AI model full terminal and file access risks accidental system deletion.
- *Solution:* Created protected directory bounds (_PROTECTED_OS_DIRS) and blocked hazardous regex patterns (format c:, wiping C:\Windows) while unlocking free access across user directories on C: and D:.

---

4. Cheat Sheet for Mentor Questions

Question	Your Ideal Answer
"Why Python and PyQt6 instead of web technologies?"	*"Desktop assistants require direct low-level OS access (audio streams, Windows APIs, registry, PowerShell, processes). Python and PyQt6 provide native hardware control and low resource overhead without the memory footprint of Electron."*
"How does the model decide when to trigger an action?"	*"Gemini uses function calling. Every action exports a JSON schema describing its parameters. Gemini evaluates user intent against these schemas and emits structured tool calls, which my action dispatcher executes and returns results back into the conversation."*
"Is user data sent to third-party servers?"	*"Only the active conversation audio goes to Google's Gemini Live API. Wake-word detection, system metrics, hardware control, and file processing are handled 100% locally on the device."*
"What makes this different from Siri or Cortana?"	*"Traditional assistants are conversational chatbots layered on top of speech engines. BRAVO is an **agentic system**: it can see your screen, run scripts, execute complex protocols, and interact with the filesystem natively."*